15,476,030 members

3.7K

articles Q&A forums stuff lounge ?

Search for articles, questions,

Watch

An All VB.NET Explorer Tree Control with ImageList Management

Jim Parsells

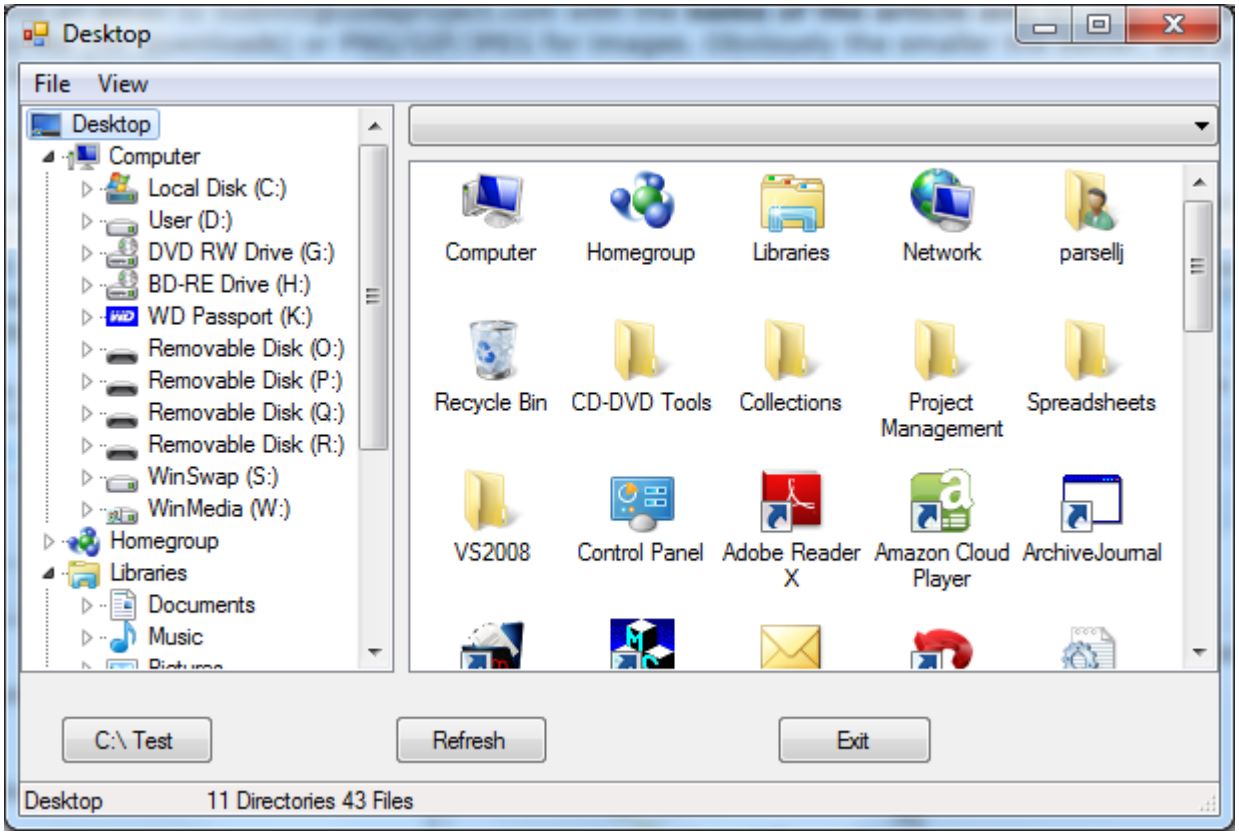
17 May 2012 CPOL

Rate me: 4.87/5 (138 votes)

Explorer TreeView control with Shell Folder access class and Icon management.

Download demo project - 45 KB

Download source - 228 KB



Introduction

The `ExpTree` control is a Windows Explorer-like TreeView control. It displays all proper icons, with overlays as appropriate. All Windows folders, including Virtual Folders like Desktop, My Computer, and History are properly displayed and made available to the containing form. The control is packaged with and uses an optimized image list management class that provides both Small and Large Icon image lists for application use. The control is just the visual aspect of a powerful Class Library(`ExpTreeLib`) that provides functionality over and above that of a combination of the `DirectoryInfo` and `FileInfo` classes. As pictured above, `ExpTreeLib` can easily be used to create a Windows Explorer-like ListView coupled with `ExpTree`.

Although .Net's `FolderBrowserDialog` is a useful substitute in many cases, `ExpTree` is a true control that may be manipulated like any other control on a Windows Form. It has a well defined interface which provides Selected Node change notification to the Form and allows both Design-time and Run-time manipulation of key aspects of the displayed Tree.

I distribute this code as a Visual Studio 2005 Solution which may be upgraded without error to VS2008 and/or VS2010 by the Visual Studio Upgrade Wizard. It targets Framework 2.0.

Version Overview

There are two supported versions of the overall Class Library `ExpTreeLib`. Version 2.12 which is described in and downloadable from this article, and Version 3.00. Version 2.12 provides a largely

static view of the Windows Shell Namespace, including the File System. Version 2.12 is a enhanced version of the package referred to as the "Rollup" version as discussed in the forum.

Version 3.00, which will be described in a soon to be written article, provides a **dynamic** view of that Namespace and adds a number of features. It is an enhanced version of the package mentioned in the forum as the "unpublished" version. Both versions have developed a community of users over the years that they have been available. Version 2.12 is useful for those applications which do not require a dynamic view or the additional features of version 3.00 and is easier to understand and use. **This article provides the basic documentation for either version**.

This version (2.12) provides a current view of TreeNodes whenever they are expanded or selected, or changed via Drag and Drop to/from the control. Changes to the file system made outside of the control are not reflected until the changed node is expanded or selected. This version supports a version of Drag and Drop which is discussed in part 2 of this article, Adding Drag and Drop to an Explorer Tree Control. A fundamental difference between version 2.12 and version 3.00 is how Drag and Drop is implemented and how it is illustrated in that version's Demo Forms.

Relative to Version 2.11, Version 2.12 has changes required by Windows Vista/Window 7, other bug fixes, and additional optimization. Applications which reference large Folders on Remote systems, which gave acceptable performance on XP system could, in Version 2.11, become very slow on Vista/Windows7. Version 2.12 fixes that for most common applications. In some cases, Version 3.00 is required to restore performance.

All previously added features and bug fixes are included. See *History* for details of this and previous updates.

Intended Audience

I have written the article and the code with an audience of developers in mind. I expect that the audience will look at the code and try it out. I have attempted to keep the comments up to date. I am interested in any constructive comments that may lead to improvements in the library.

Background

My design goals were to create a control that only needed one *.dll* (no auxiliary wrapper *.dll*s), showed the correct icons on any Windows system, would work with Virtual Folders as well as FileSystem folders, was quick, and used few resources. Since the rest of my code was to be in VB.NET, I wanted the control to be written in VB.NET. I could not find any code on this or any other site that met my requirements. Almost all other similar controls were written in C#, and none fully met the other requirements.

Controls based on `DirectoryInfo` and `FileInfo` classes will not handle Virtual Folders. Controls based on adding a reference to *Shell32.dll* require an extra *.dll* to wrap the COM interface and will not report hidden files and directories. Applications using either approach require additional classes to deal with

icons since neither gets icon information. Since I had written Shell-accessing *.dll*s in C, I was familiar with the techniques, so I decided to attack the problem using the `IShell` Folder Interface with `SHGetFileInfo` providing the icon information.

Class Overview

The control, `ExpTree`, is packaged with several supporting classes into one library assembly and *.dll* (`ExpTreeLib`). `ExpTreeLib` contains these classes:

`ShellDll`	API declarations, interfaces, structures, enumerations, and constants.
`CShItem`	The main class of this library.
`SystemImageListManager`	A class to manage Large and Small System image lists.
`ExpTree`	The actual control.

Use of the `SystemImageListManager` class is optional, however, `ExpTree` initializes and uses it. If the application needs to display FileSystem icons, the class will provide them. See the SystemImageList Manager Class section below, for details.

Details of the `CShItem` class are discussed below. It wraps a collection of information that describes one folder or file. In use, it is similar to a `DirectoryInfo` or `FileInfo` instance. However, it is built using the Shell's `IShellFolder` interface, and therefore, can represent all folder and file types available on the system.

The library also contains other classes solely for Drag and Drop support which will not be discussed here.

Using the Control

To use the control, add a reference to its *.dll* to your project, and then add the control to the ToolBox. To add the control to the ToolBox, right click on the ToolBox, click Customize ToolBox, and then browse for the DLL. Once you have done this, you may use it like any other control. In addition to the normal `UserControl` properties, the `ExpTree` control exposes several Properties:

`AllowDrop`	Design and Run Time	Allow/Disallow Drop on Tree
`ShowHiddenFolders`	Design and Run Time	Show/Hide hidden folders
`ShowRootLines`	Design and Run Time	Allow/Disallow collapse of TreeRoot
`StartupDirectory`	Design and Run Time	Select root directory of Tree
`RootItem`	Run-Time only	Set root to a specific `CShItem`

`SelectedItem`	Run-Time only	Gets the currently selected `CShItem`

`StartupDirectory` sets the root of the TreeView. It will only accept a `SystemFolder` as a startup directory. The most useful ones are `Desktop` and `My Computer`. Change the `StartUpDirectory` at design time to see, in the IDE, what the initial display will be.

`RootItem` is a run-time only property which is used to reset the tree root to another folder which may be any folder available in the TreeView.

Hint

To set an `ExpTree` to appear to start Rooted in some non-System Folder:

1. In the IDE, set the `StartupDirectory` to the Desktop.
2. In the Form's `Load` event, set the `RootItem` to the desired Folder, as in:

VB

```
ExpTree1.RootItem = CShItem.GetCShItem("C:\MyAppData")
```

ExpTree Methods

Method	Type	Remarks
`RefreshTree`	N/A	Rebuild tree through `SelectedNode`
`ExpandANode`	Boolean	Expands tree through input `Path` or `CShItem`

The methods `RefreshTree` and `ExpandANode` are not needed for basic usage and are discussed later.

ExpTree Events

`StartUpDirectoryChanged`	Used for design-time interaction
`ExpTreeNodeSelected`	Raised when `TreeNode` is selected

The `EventArgs` for `ExpTreeNodeSelected` are a string containing the full path of the underlying folder and the `CShItem` representing the `SelectedNode`.

Assume you have a form with an `ExpTree` named `ExpTree1`, a `ListView` named `lv1`, and a `StatusBar` named `sbr1`. To use the control, you must Import a few items:

VB

```
Imports ExpTreeLib
Imports ExpTreeLib.CShItem
Imports ExpTreeLib.SystemImageListManager
```

```
Public Class frmExplorerLike
   Inherits System.Windows.Forms.Form
```

In the Form `New` routine, set up to use the image lists:

VB

```
'Add any initialization after the InitializeComponent() call
SystemImageListManager.SetListViewImageList(lv1, True, False)
SystemImageListManager.SetListViewImageList(lv1, False, False)
```

The `SetListViewImageList` statements set the `ListView`'s `LargeImageList` and `SmallImageList` to be the corresponding System image lists.

Add the following event handler:

VB

```
Private Sub lv1_VisibleChanged(ByVal sender As Object, _
        ByVal e As System.EventArgs) Handles lv1.VisibleChanged
    If lv1.Visible Then
        SystemImageListManager.SetListViewImageList(lv1, True, False)
        SystemImageListManager.SetListViewImageList(lv1, False, False)
    End If
End Sub
```

To change the contents of the `ListView` when a node is selected in `ExpTree1`, declare an event handler as follows:

VB Shrink ▲

```
Private Sub AfterNodeSelect(ByVal pathName As String,
        _ ByVal CSI As CShItem) Handles
    ExpTree1.ExpTreeNodeSelected Dim dirList As New
    ArrayList() Dim fileList As New
    ArrayList() Dim TotalItems As
    Integer If CSI.DisplayName.Equals(CShItem.strMyComputer) Then
        'avoid re-query since only has dirs
        dirList = CSI.GetDirectories
    Else
        dirList = CSI.GetDirectories
        fileList = CSI.GetFiles
    End If
    TotalItems = dirList.Count + fileList.Count
    If TotalItems > 0 Then
        Dim item As CShItem
        dirList.Sort()
        fileList.Sort()
        Me.Text = pathName
        sbr1.Text = pathName & "                  " & _
                    dirList.Count & " Directories " & _
                    fileList.Count & " Files"
        Dim combList As New ArrayList(TotalItems)
        combList.AddRange(dirList)
```

```
        combList.AddRange(fileList)

        'Build the ListViewItems & add to lv1
        lv1.BeginUpdate()
        lv1.Items.Clear()
        For Each item In combList
            Dim lvi As New ListViewItem(item.DisplayName)
            With lvi
              '
              ' SubItem formatting and adding to lvi omitted from
              '    article text
              '
              'Set ListViewItem's IconIndex
              '(and add Icon to lists if necessary)
                .ImageIndex = _
                 SystemImageListManager.GetIconIndex(item, False)
                .Tag = item
            End With
            lv1.Items.Add(lvi)
        Next
        lv1.EndUpdate()
    Else
        sbr1.Text = pathName & "Has No Items"
    End If
End Sub
```

The test for "`strMyComputer`" at the beginning uses the local system string for "My Computer" to avoid the annoying re-query of the *A:* drive each time "My Computer" is selected. Some special handling is required in order to get the folders and files sorted as Windows Explorer does. That special handling is done in `CShItem`'s `IComparable.CompareTo` routine which is called when we sort the Directory and File `ArrayList`s.

Icon fetching and adding to both image lists is handled by the call to `GetIconIndex`. `GetIconIndex`'s second parameter is set to `False` to indicate that the "Open" `IconIndex` should not be fetched.

Note: the download demo obtains and sets the icon indices in a separate thread. The code shown above is a simplified, single thread approach.

SystemImageListManager Class

This class is based on System image lists. It accesses two System image lists, one containing small icons, and one with large icons. The lists are synchronized such that the same `IconIndex` refers to the same icon in each list. When queried for an `IconIndex` of a `CShItem`, it determines if the icon should have an overlay and, if so, adds the icon with overlay as an additional icon in the System image lists. Since the `IconIndex` reported by `SHGetFileInfo` is not necessarily the actual `IconIndex` for the folder or file (which may need to use the icon plus overlay version), I use a `HashTable` to store the actual `IconIndex`. The `HashTable` key is based on the reported `IconIndex`, modified to reflect any additional overlays. In actual practice, the number of icons stored in the System image lists and the size of the `HashTable` is fairly small.

The `SystemImageListManager` class contains only `Shared` properties and methods. Since it is managing an external resource (two System image lists), only `Shared` properties and methods are needed or appropriate. *Do **not** modify these two System image lists in any fashion outside of* `SystemImageListManager`.

Any of `SystemImageListManager` properties or methods will call the class' `Initializer` routine.

SystemImageListManager Initializer

VB

```
Private Shared Sub Initializer()
    If m_Initialized Then
        Exit Sub
    End If
    Dim dwFlag As Integer = SHGFI.USEFILEATTRIBUTES Or _
                    SHGFI.SYSICONINDEX Or _
                    SHGFI.SMALLICON
    Dim shfi As New SHFILEINFO()
    m_smImgList = SHGetFileInfo(".txt", _
                        FILE_ATTRIBUTE_NORMAL, _
                        shfi, _
                        cbFileInfo, _
                        dwFlag)
    If m_smImgList.Equals(IntPtr.Zero) Then
        Throw New Exception("Failed to create Small ImageList")
    End If
    '
    ' Identical code as above using SHGFI.LARGEICON
    '   Omitted from Article text ... see source code
    '
    m_Initialized = True
End Sub
```

`Initializer` checks that this is the first call. If not, it assumes all is set up. If the first call, it obtains the `Handle` of a small and a large System image list, checking for success each time.

SystemImageListManager Properties

`SmallList`	The Handle of the small `ImageList`
`LargeList`	The Handle of the large `ImageList`

These `ReadOnly` properties may be of use to the application. The demo makes no use of them except internal to the class.

SystemImageListManager Methods

GetIconIndex

VB

```
Public Shared Function GetIconIndex(ByRef item As CShItem, _
                      Optional ByVal GetOpenIcon As Boolean = False, _
                      Optional ByVal GetSelectedIcon As Boolean = False _
                      ) As Integer
```

`GetIconIndex` returns the `IconIndex` in both System image lists of the icon needed for the `CShItem` item. The `Optional` parameter `GetOpenIcon` instructs `GetIconIndex` to return the "Open" icon for the `CShItem` rather than the "Normal" icon. The `Optional` parameter `GetSelectedIcon` requests the "Selected" icon.

Internally, `SystemImageListManager` maintains a `HashTable` whose `Key` is based on the System imagelist `IconIndex`, the Link and Shared states of the referenced `CShItem`, and the "Open" or "Selected" state of the icon. The `Value` stored in the `HashTable` is the `IconIndex` of the icon in the System image lists.

If the `IconIndex` is already known (in the `HashTable`), the function simply returns the `HashTable Value` as the function value.

If the desired icon is not known (in the `HashTable`) and if the icon will contain overlays, the function obtains the icon using `SHGetFileInfo`, stores it in the System image lists, enters the `IconIndex` into the `HashTable`, and returns the `IconIndex` and returns it as the function value.

If the desired icon does not have overlays, then the `IconIndex` already stored in the `CShItem` is the correct `IconIndex`. In this case, the function stores the `IconIndex` into the `HashTable`, and returns it as the function value.

GetIcon

VB

```
Public Shared Function GetIcon(ByVal Index As Integer, _
       Optional ByVal smallIcon As Boolean = False) _
       As Icon
```

Returns a GDI+ copy of the Large (default) or Small icon from the imagelist at the specified `Index`.

SetListViewImageList

VB

```
Public Shared Sub SetListViewImageList( _
      ByVal listView As ListView, _
      ByVal forLargeIcons As Boolean, _
      ByVal forStateImages As Boolean)
```

This method attaches the appropriate System image list to the `ImageList` of a `ListView`. The `forLargeIcons` parameter selects which list to attach to which (`True` for large, `False` for small). I have not done anything with the `forStateImages` parameter except to pass it as `False`, always.

SetListTreeViewImageList

VB

```
Public Shared Sub SetTreeViewImageList( _
    ByVal treeView As TreeView, _
    ByVal forStateImages As Boolean)
```

This method attaches the appropriate System image list to the `ImageList` of a `TreeView`. I have never tested or used the `forStateImages` parameter, except to set it to `False`.

Both of these methods use the `SendMessage` API to send a message to the control, attaching the System image list as the control's `ImageList`. See the source code for details. Note that .NET is not aware of this attachment. If the control is hidden and then shown, the attachment must be reestablished in a `VisibleChanged` event handler.

CShItem Class

Version 2 Changes

The `CShItem` class is the main class of the ExpTree library. Additions to any part of the library generally require changes to it. The major change between version 2 and previous versions actually occur in `CShItem`. Each `CShItem` that represents a folder maintains an `ArrayList` of `CShItem`s representing the sub-folders that it contains. In versions prior to version 2, that `ArrayList` was never updated. If the method `GetDirectories` was called with the parameter `Optional Refresh As Boolean = False` set to `True`, the entire `ArrayList` was discarded and recreated. In version 2, the parameter `Optional doRefresh As Boolean = True` instructs `GetDirectories` to call a new function, `RefreshDirectories`. `RefreshDirectories` checks for changes and creates new `CShItem`s for added directories, and deletes the `CShItem` representing directories that no longer exist. This process, implemented by other new code, is done in a low-cost fashion, so that this directory refresh can be done frequently. In ExpTree, it is done at every TreeNode expand, every select, and in several other cases relating to Drag and Drop.

Note that `CShItem`s representing Files are not kept, but regenerated at each `GetFiles` or `GetItems` request. A potential line of study is to look at the memory versus processor time tradeoffs involved in treating file `CShItem`s similar to directory items.

Version 3.00 of this class is completely different in its approach.

Constructors

Version 2 depreciates the use of `New` as a method of obtaining `CShItem`s.

In version 2, the `Sub New(ID As CSIDL)` and `Sub New(path As String)` routines are still supported. However, Version 3.00 does not support any Public Sub New. For Version 2, the preferred replacement functionality is provided by the `GetCShItem` routines described here:

- `GetCShItem(ByVal ID As CSIDL) As CShItem`

  `CSIDL` is an `Enum` representing System Special Folders, declared in the `ShellDll` class. `ExpTree` defines a subset of those as valid for its purposes. Unlike the equivalent `Sub New`, there is no restriction on which `CSIDL` may be used, beyond that of simple availability on a particular OS. Usage is illustrated by this code fragment which obtains the `CShItem` for *My Computer*.

  VB

  ```
  Dim special As CShItem
     special = GetCShItem(CSIDL.DRIVES)
  ```

- `GetCShItem(ByVal path As String) As CShItem`

  `path` is a valid directory path (for example, "*C:*\ "). Path can be any `CShItem.Path` property, including GUIDs. A simple use is illustrated by this code fragment which obtains the `CShItem` for a specific Directory:

  VB

  ```
  Dim special As CShItem
     special = GetCShItem("C:\Temp\Test")
  ```

Properties

Property	Type	Remarks
`DisplayName`	`String`	Display name
`Path`	`String`	Full path (see note 1)
`TypeName`	`String`	Type of item (see note 2)
`FullName`	`String`	Full name of items (see note 3)
`IconIndexNormal`	`Integer`	Index into `SystemImageList`
`IconIndexOpen`	`Integer`	Index into `SystemImageList`
`HasSubFolders`	`Boolean`	May have sub-folders
`IsBrowsable`	`Boolean`	Can be browsed in place
`IsDropTarget`	`Boolean`	Items can be dropped here

Property	Type	Remarks
`IsFileSystem`	`Boolean`	Is part of file system
`IsFolder`	`Boolean`	Is a folder
`IsDisk`	`Boolean`	Is a disk
`IsLink`	`Boolean`	Is a shortcut
`IsRemovable`	`Boolean`	Is a removable device
`IsReadOnly`	`Boolean`	Is `ReadOnly`
`IsShared`	`Boolean`	Is `Shared`
`IsSystem`	`Boolean`	Is a System file
`LastWriteTime`	`DateTime`	See `FileInfo` documentation
`LastAccessTime`	`DateTime`	See `FileInfo` documentation
`CreationTime`	`DateTime`	See `FileInfo` documentation
`Length`	`Long`	Size in bytes of a file
`CanCopy`	`Boolean`	Item can be copied
`CanDelete`	`Boolean`	Item can be deleted
`CanLink`	`Boolean`	Item can have a link created for it
`CanMove`	`Boolean`	Item can be moved
`PIDL`	`IntPtr`	Usable in `SHGetFileInfo`
`clsPidl`	`cPidl`	A class for manipulating `PIDL`s as `Byte()`
`strMyComputer`	`String`	"My Computer" on this computer
`strSystemFolder`	`String`	"System Folder" on this computer
`DesktopDirectoryPath`	`String`	The path of user's Desktop directory

All properties are `ReadOnly`.

Note 1: For File System objects, the `FullPath` property is just that, the full path. For non-File System objects, the full path may be a GUID.

Note 2: `TypeName` is the type name reported by `SHGetFileInfo`.

Note 3: `FullName` is usually the same as `DisplayName`. However, in the case of *.lnk* files, `DisplayName` does not include the *.lnk* extension. `Fullname` does. Given a link file whose `Path` is "*C:\Temp\ABC.txt.lnk*", `Displayname` will return "*ABC.txt*", `FullName` will return "*ABC.txt.lnk*".

The `IconIndex...` properties report the base `IconIndex` into the System image list. This is not directly useful to applications unless only non-overlay icons are desired.

In almost all cases, `PIDL` should not be used. It is visible only because `SystemImageListManager` needs to refer to it. `PIDL` may be useful if the application needs to call certain Shell *.dll*s. The `clsPidl` property is an instance of the `cPidl` class. It exposes methods of examining the `PIDL` as a `Byte()`. See the source code for further information.

The `...Time` properties and the `Length` property are exactly the same as returned by the `FileInfo` class. I cheat and create a `FileInfo` instance to retrieve these values when any one of them is requested.

The `str...` properties provide the strings that represent "My Computer" and "System Folder" on the computer running the application. This provides a Locale neutral method of testing for these special names. The special name "Desktop" is provided by the `DisplayName` of the `CShItem` returned by `GetDeskTop` and is also Locale neutral.

Methods

Method	Return Type	Return Value
Shared Method		
`GetCShItem`	`CShItem`	See above for description
`GetDeskTop`	`CShItem`	The Desktop
Instance Methods		
`GetDirectories`	`ArrayList` of `CShItem`s	All folders in a `CShItem`
`GetFiles`	`ArrayList` of `CShItem`s	All files in a `CShItem`
`GetItems`	`ArrayList` of `CShItem`s	All files and folders in a `CShItem`
`RefreshDirectories`	`Boolean`	`True` if any changes were made.
`ToString`	`String`	`DisplayName`
`DebugDump`	None	Writes info to the Debug console

`GetDeskTop` returns the one and only `CShItem` of the Desktop. The class maintains this `CShItem` internally, building it when the class is first accessed in any way. `GetDeskTop` returns the actual `CShItem`, not a clone.

`GetDirectories`, `GetFiles`, and `GetItems` return an `ArrayList` of `CShItem`s as requested. If there are none of the requested types in a folder, they return an empty `ArrayList`. If the `CShItem` represents a file, an empty `ArrayList` is returned. An empty `ArrayList` is also returned on common error conditions, for example, a Not Ready disk (an empty CD drive, for example). Unlike Windows Explorer, the class does not post an Abort-Retry message box in those cases. Previous versions, before version

2, would throw an exception for unexpected errors occurring in the internal routine called by these methods, *only when compiled in Debug mode*. Version 2 and above will no longer throw an exception, returning an empty `ArrayList` on any error condition.

`RefreshDirectories` ensures that the `ArrayList` returned by `GetDirectories` reflects the current state of the file system. It returns `True` if there were any changes. `RefreshDirectories` is called by `GetDirectories` (unless specifically instructed not to by an `Optional` parameter), so there is seldom a need to call it directly.

ExpTree Control

Finally, we get to the control itself. Given the `CShItem` and `SystemImageListManager` classes, the control is fairly simple.

ExpTree Properties and Events

Property	
`AllowDrop`	Allows (`True`) or Prevents (`False`) Drops onto the Tree.
`StartUpDirectory`	Must be a `CSIDL` for a special folder.
`RootItem`	Sets the root of the Tree to a `CShItem`.
`SelectedItem`	Returns `CShItem` of current `SelectedNode`.
`ShowHidden`	Allows/Disallows the showing of hidden directories in the TreeView.
`ShowRootLines`	Allows/Disallows a line and expansion/compression box to be shown in the TreeView.
Events	
`ExpTreeNodeSelected`	Raised when a `TreeNode` is selected.
`StartUpDirectoryChanged`	Raised when the `StartUpDirectory` property is set.
Methods	
`ExpandANode`	Expands tree through the node representing the input `CShItem`. Returns `False` on failure to expand.
`RefreshTree`	Reinitializes tree and expands it through the previously selected node.

`StartUpDirectory` is a `CSIDL` representing a System Special Folder. The list of folders that the control can deal with is available to the IDE via `ExpTree`'s Property Sheet.

`RootItem` is a Run-Time only property. Setting this Item via a run-time call results in re-setting the entire tree to be rooted in the input `CShItem`. The `CShItem` must be a valid `CShItem` of some kind of folder (File Folder or System Folder). Attempts to set it using a non-folder `CShItem` are ignored. Usage of this property is illustrated in the demo in the `ListView`'s `MouseUp` event and in the code behind the "*C:\ Test*" button.

`ExpTreeNodeSelected` is the event fired when the `TreeView`'s `AfterSelect` event occurs. This notifies the containing `Form` of the event. The `Event` signature is:

VB

```
Public Event ExpTreeNodeSelected(ByVal SelPath As String, _
             ByVal Item As CShItem)
```

where `Item` is the `CShItem` representing the selected node, and `SelPath` is the path of that `CShItem`. In the case of Virtual Folders, where the path is a GUID, `SelPath` contains the `DisplayName` of the `CShItem`.

`StartUpDirectoryChanged` is the event fired when the initial directory is set. It is `Public` in case the containing `Form` needs notification of this event. Normally, this is not needed since a change to the root of the Tree always selects the new root and thus fires the `ExpTreeNodeSelected` event.

`ExpandANode` is a revision of the `ExpandANode` originally presented in the forum for this article. Internally, this method is very different from the original and is not limited to File System directories as was the original. Any `CShItem` may be used as the input path. Unlike the original, this version will not force the Tree to be rooted on either the `Desktop` or

```
My
Computer
```

. This version leaves the original Tree root in place. The method expands the tree from the tree root, expanding nodes as necessary through the input `CShItem`. Its signature is:

VB

```
Public Function ExpandANode(ByVal newItem As CShItem) As Boolean
```

The method returns `True` if the expansion was successful, `False` otherwise. The class provides an alternate `ExpandANode` which takes a `Path` as its argument. The alternate signature method calls `GetCShItem`, checks the return, and calls the other `ExpandANode` with the returned `CShItem`.

`RefreshTree` is a method which causes the entire tree to be recreated and then expanded down to the original (prior to `RefreshTree` call) selected node. This allows the Tree to reflect changes made to the directory structure external to the control. If the originally selected node is no longer valid, for example, it and/or some earlier part of its path were deleted or renamed, the tree is expanded

through the lowest valid point in its original path. This method's code is almost identical to the code presented by Calum McLellan in the forum. One difference is that it now defaults to rooting the Tree in the original Tree root rather than defaulting to the `Desktop`. Another difference is that it suppresses the raising of `ExpTreeNodeSelected` events until the refresh has completed. This method benefits from the new version of `ExpandANode` in that it is no longer limited to dealing only with File System directories.

The signature of this method is:

VB

```
Public Sub RefreshTree(Optional ByVal root As CShItem = Nothing)
```

The optional parameter `root` allows for dynamic resetting of the Tree root as part of the refresh operation.

ExpTree Code

In the initialization of `ExpTree`, we set the `TreeView`'s `ImageList` and add the control's handler for changes to `StartUpDirectory`.

VB

```
'Add any initialization after the InitializeComponent() call
  SystemImageListManager.SetTreeViewImageList(tv1, False)
  AddHandler StartUpDirectoryChanged, AddressOf OnStartUpDirectoryChanged
  OnStartUpDirectoryChanged(m_StartUpDirectory)
```

The `Public Property StartUpDirectory` starts the work when the `StartUpDirectory` is set or changed:

VB

```
Private m_StartUpDirectory As StartDir = StartDir.Desktop

<Category("Options"), _
 Description("Sets the Initial Directory of the Tree"), _
 DefaultValue(StartDir.Desktop), Bindable(True)> _
   Public Property StartUpDirectory() As StartDir
        Get
           Return m_StartUpDirectory
        End Get
        Set(ByVal Value As StartDir)
        If Array.IndexOf(Value.GetValues(Value.GetType), _
         Value) >= 0 Then
            m_StartUpDirectory = Value
            RaiseEvent StartUpDirectoryChanged(Value)
        Else
            Throw New ApplicationException( _
            "Invalid Initial StartUpDirectory")
        End If
```

```
    End Set
End Property
```

The property attributes give the designer information. The code at `If Array.IndexOf...` compares the input `Value` with the `Enum`'s allowable values and `Throw`s an exception if not valid. If valid, the private version of the property is set and a `StartUpDirectoryChanged Event` is raised.

The real work is done in the `OnStartUpDirectoryChanged` event handler:

VB Shrink ▲

```
 Private Sub OnStartUpDirectoryChanged(ByVal newVal As StartDir)
   If Not IsNothing(Root) Then
       ClearTree()
   End If
   Dim L1 As ArrayList
   Dim special As CShItem
   special = GetCShItem(CType(Val(m_StartUpDirectory), ShellDll.CSIDL))
   Root = New TreeNode(special.DisplayName)
   BuildTree(special.GetDirectories)
   Root.ImageIndex = SystemImageListManager.GetIconIndex(special, _
    False)
   Root.SelectedImageIndex = Root.ImageIndex
   Root.Tag = special
   tv1.Nodes.Add(Root)
   Root.Expand()
End Sub

Private Function BuildTree(ByVal L1 As ArrayList)
  L1.Sort()
  Dim CSI As CShItem
  For Each CSI In L1
      If Not (CSI.IsHidden And Not m_showHiddenFolders) Then
          Root.Nodes.Add(MakeNode(CSI))
      End If
  Next
End Function

Private Function MakeNode(ByVal fi As CShItem) As TreeNode
  Dim newNode As New TreeNode(item.DisplayName)
  newNode.Tag = item
  newNode.ImageIndex = SystemImageListManager.GetIconIndex(item, False)
  newNode.SelectedImageIndex = SystemImageListManager.GetIconIndex(item, True)
  If item.IsRemovable Then
      newNode.Nodes.Add(New TreeNode(" : "))
  ElseIf item.HasSubFolders Then
      newNode.Nodes.Add(New TreeNode(" : "))
  ElseIf item.GetDirectories.Count > 0 Then
      newNode.Nodes.Add(New TreeNode(" : "))
  End If
  Return newNode
End Function

Private Sub ClearTree()
  tv1.Nodes.Clear()
```

```
  Root = Nothing
End Sub
```

First, the folders of the base are fetched and sorted. For each folder in the base, we create a new `TreeNode` with the correct icons, and add it to the Root node. Note that each `TreeNode's` `Tag` is set to the `CShItem` that it belongs to. If the sub-folder may have sub-folders of its own, we create a dummy node and add it to the sub-node, so the `Treeview` will show a "+" and allow expansion.

The code in `BuildTree` that checks `.IsHidden` prevents Hidden directories from being shown in the `TreeView` if the `ShowHiddenFolders` property is `False`. The `If ... ElseIf` sequence in `MakeNode` avoids checking floppy drives so as to prevent the annoying floppy access. It also works around the fact that a directory with all hidden members will be reported by `.HasSubFolders` as `False`. Finally, we attach the Root node to the `TreeView` and `Expand` the root to get the final display.

The `BeforeExpand Event` of the `Treeview` is very similar to the code described above. The interesting part is:

VB

```
Private Sub tv1_BeforeExpand(ByVal sender As Object, _
    ByVal e As System.Windows.Forms.TreeViewCancelEventArgs) _
    Handles tv1.BeforeExpand
    Dim oldCursor As Cursor = Cursor
    Cursor = Cursors.WaitCursor
    If e.Node.Nodes.Count = 1 AndAlso _
     e.Node.Nodes(0).Text.Equals(" : ") Then
        e.Node.Nodes.Clear()
        Dim CSI As CShItem = e.Node.Tag
        Dim D As ArrayList = CSI.GetDirectories(m_refresh)
        If D.Count > 0 Then
            '.... processing steps omitted
        End If
    Else
        RefreshNode(e.Node)
    End If
    Cursor = oldCursor
End Sub
```

If the node is a dummy node, then clear it and process it similar to the code described above.

Otherwise, assume that the sub-nodes have already been set up and call `RefreshNode`, ensuring that the content matches reality. Note that if there are no sub-folders for this node, then the `TreeNode` will be cleared, removing the "+" and preventing future expansion.

Lastly, we have the `AfterSelect Event` which passes the `CShItem` from the `SelectedNode` to the containing `Form`.

VB Shrink ▲

```
Private Sub tv1_AfterSelect(ByVal sender As System.Object, _
        ByVal e As System.Windows.Forms.TreeViewEventArgs) _
```

```
        Handles tv1.AfterSelect
    Dim node As TreeNode = e.Node
    Dim CSI As CShItem = e.Node.Tag
    If CSI Is Root.Tag AndAlso Not tv1.ShowRootLines Then
        With tv1
            .BeginUpdate()
            .ShowRootLines = True
            RefreshNode(node)
            .ShowRootLines = False
            .EndUpdate()
        End With
    Else
        RefreshNode(node)
    End If
    If EnableEventPost Then 'turned off during RefreshTree
        If CSI.Path.StartsWith(":") Then
            RaiseEvent ExpTreeNodeSelected(CSI.DisplayName, CSI)
        Else
            RaiseEvent ExpTreeNodeSelected(CSI.Path, CSI)
        End If
    End If
End Sub
```

The `SelectedNode` is updated by `RefreshNode`. The test for `ShowRootLines` works around a display problem that arises when the tested condition is `True`. If event posting has not been suppressed for `RefreshTree`, then raise the `ExpTreeNodeSelected` event, passing the `CShItem` and the path of the node. Note that some System Folders' path is a GUID. In that case, we return the `DisplayName` of the `SelectedNode` rather than the path. The true path is still available in the `CShItem`.

The Demo Program and other thoughts

The demo Forms do no useful work except illustrating the usage of the control and classes presented here. I really wasn't trying to duplicate Windows Explorer. There are two Forms in the Demo package. `frmExplorerLike` (shown above and described here) exists only to illustrate how to use some of the methods available in ExpTreeLib. `frmDragDrop` is a bit more realistic and and illustrates Drag From and Drop To ExpTree as well as Drag From the ListView.

Left-click a folder in the `ListView` to cause the corresponding folder in the Tree to be expanded and that folder's contents to be displayed in the ListView.

`frmExplorerLike` shows three run-time methods of changing the root of the Tree. Right-clicking a folder in the `ListView` will cause that folder to become the new Tree root. It also will fill the `ComboBox` with the names of the parents of that folder. Selecting one of the entries in the combobox will set the Tree root to that folder. In other words, it provides a way to get back to the original Tree root.

Clicking the "*C:\ Test*" button will cause the Tree root to become *C:*. I provide no way to navigate back to the original Tree root in this case. Given the `RootItem` property of `ExpTree` and the `GetCShItem(Path as String)` method of `CShItem`, the code to accomplish this change in the demo program is trivial.

VB

```
Private Sub cmdCTest_Click(ByVal sender As System.Object, _
 ByVal e As System.EventArgs) Handles cmdCTest.Click
    Dim cDir As New CShItem = GetCShItem("C:\")
    If cDir.IsFolder Then
        ExpTree1.RootItem = cDir
    End If
End Sub
```

Click the Refresh button on `frmExplorerLike` and `RefreshTree` will be invoked. You may test this feature by creating a test directory tree, running the demo, navigating to the bottom of the test tree, deleting some or all of the test directory tree via Windows Explorer, and then clicking the Refresh button.

Both Demo Forms now gather icon indices, for display in its `ListView`, in a separate thread which improves initial startup and general responsiveness. This is not shown is the code presented in this article. See the demo for the actual code.

Feedback from readers of this article has been very helpful in making this control better. A look at the History section and the forum will show that multiple feature additions and bug fixes came as a direct result of that feedback. Thanks everyone.

Credits

My original article contained a class for accessing System image lists. Some important fragments of that class survive in `SystemImageListManager`. The original was simply a translation from C# to VB.NET of some of Steve McMahon's System image list class which may be found here. Steve's class has substantial additional capabilities for drawing icons and attaching them to other types of controls.

Calum McLellan has made significant contributions that improved this control. Calum's article **Explorer ComboBox and ListView in VB.NET** extends this library with both `ComboBox` and `ListView` classes.

History

- 04/20/2012 -- Version 2.12.1 Updated Downloads to include Windows Explorer style sorting to the displayed File/Folder names. This change added a new class (`StringLogicalComparer`) and a one line change to CShItem to use that Class. Result is: xx11xx sorts before xx101xx.
- 04/20/2012 -- Version 2.12. Update of Downloads and article to include:
    - ASUS fix which also probably applies to Carbonite and several other Cloud backups that are implemented as Shell Extensions.
    - Proper handling of Zip and other Compressed files on Vista/Win7 (ensure they are treated a Files, not Folders)
    - Proper handling of `AllowDrop` property in `ExpTree`. Can now be usefully set in IDE.

  - Changed the definition and source of `CShItem.Attributes` such that this property now is set from a `FileInfo` or `DirectoryInfo`. The definition is now a `System.IO.FileAttributes` (as it always should have been).
  - Made `CShItem.HasSubfolders` a fill on demand property, avoiding the cost of retrieval when not needed. A big win in some cases.
  - Modified the handling of the `HasSubFolders` attribute to compensate for difference between XP and Vista/Win7. On Vista/Win7 client systems this is a dramatic improvement in responsiveness.
  - Minor changes to eliminate some harmless compiler Warnings.
  - Removal of dead and debug code and correction of some comments.

- 03/12/2006 -- Version 2.11. Updated to support VS2005. Deleted the `Application.DoEvents` call in `CShItem.GetContents` as discussed in the forum.
- 09/16/2005 -- Version 2.1. Update to source and demo to make equal to same files in Part 2 of this article. Minor fix to this article's code, larger fix to code covered in Part 2.
- 08/23/2005 -- Version 2 release - Update to article, source, and demo.
  - Changed directory refresh strategy to update cached directories in `GetDirectories` unless specifically prevented via an `Optional` parameter.
  - Added `CShItem.GetCShItem` to replace functionality of `Sub New(ID as CSLID)` and

    ```
    Sub New(Path
        As String)
    ```

    .
  - Added refresh of node content in `BeforeExpand` and `AfterSelect` events.
  - Added Drag and Drop -- not discussed in this article.
  - Added the properties `ShowHiddenFolders` and `ShowRootLines` to `ExpTree`.
  - In `ShellDll`, changed declaration of `POINT` from `Private` to `Public` which may break existing code. Fully specify `System.Drawing.Point` or `ShellDll.Point` as appropriate.
  - Added ability to get the selected `IconIndex` as well as the normal and open ImageIndices to `SystemImageListManager`.
  - Added ability to get a true Small Icon from `GetIcon` -- From Calum McLellan.
  - Added many additional properties and methods to `CShItem`.
  - By popular demand, removed the `Throw` of an error for certain conditions from `CShItem` when compiled under `DEBUG`.
  - Multiple small improvements, some bug fixes, along with some code reorganization.

- 04/02/2005 -- Update to source and demo.
  - Modified `ExpTree` control to paint the tree when initially dropped on a form in the IDE, and to hide from the IDE those properties that can not be changed there.
  - Changed the sort order of `CShItem`s such that "My Documents" appears before "My Computer" in the tree.

  - Added the `Public Shared` field `strMyDocuments` to `CShItem` which contains the Locale representation of the string "My Documents".
  - Added the `Public ReadOnly Property IsHidden` to `CShItem`. (Thanks Calum.)
  - Modified `SystemImageListManager` to get and use the actual Small Icon for Small Image Lists, rather than re-using the Large Icon (Thanks Calum).
  - Fixed the demo to use `SystemImageListManager` to set icon indices for the ListView. This was broken when threading was added to the demo.

- 03/02/2005 -- Update to source, demo, and article.
  - Rewrote `ExpandANode` to remove limitations of previous version.
  - Added `RefreshTree` method to allow application to force a rebuilding of the Tree to display changes to the directory structure.
  - Added `SelectedItem` property which returns the `CShItem` of the current `SelectedNode`.
  - Initialized the `HideSelection` property of the `TreeView` to `False`.
  - Modified `Sub New(path as String)` to accept any `CShItem.Path`, including GUIDs.
  - Fixed XP related problem which suppressed display of ZIP files.
  - Removed (in Release compilations only) the throwing of an exception for unexpected errors.
  - Added threading to the demo program to improve responsiveness.
- 01/11/2005 -- Update to correct bug that prevented the creation of `CShItem`s in a worker thread.
- 01/09/2005 -- Update to correct a bug and to incorporate some additional features.
  - Fixed the routines `ItemIDListSize` and `PidlCount` in `CShItem` which would fail in some very rare instances.
  - Modified `Sub New(path as String)` in `CShItem` to accept a file path as well as a directory path.
  - Added the required special handling so that My Documents can be used as a base directory. Code changes in

    ```
    Sub New(StartDir as
        CSIDL)
    ```

    to accomplish this. Also uncommented `StartDir` entry for My Documents so it can be used in the designer.
  - Improved `GetItems()` property to avoid an extra pass over the contents of a directory.
  - Added `ExpandANode` method to `ExpTree`. This is the limited version of this method discussed in the forum for this article.
  - Modified demo to clear the `ListView` when an empty directory is selected in the `TreeView`. Demo also contains some commented out code that may be used to exercise some of the added functionality. See demo source for directions on how to do this.
- 11/29/2004 -- Added features to `CShItem`, `ExpTree`, and the demo program. Small addition to `ShellDll`. Made `CShItem` and the demo more Culture neutral. Modified article to reflect changes.

    - Added a variant constructor to `CShitem` to allow the creation of a `CShItem` based on a valid directory path (e.g. -- "*C:*\").
    - Added a Run-Time only property to `ExpTree` to allow the changing of `ExpTree`'s root directory dynamically.
    - Modified demo to illustrate the new properties.
    - Removed or modified tests based on `CShItem`'s `TypeName` and `DisplayName` strings that would fail in a non-English Culture setting. Also modified a test in the demo which would fail under the same circumstance. Modified the creation code for the Desktop `CShItem` to set its path to its GUID and to obtain its `DisplayName` from `SHGetFileInfo` rather than arbitrarily setting it to "Desktop".
    - Modified `CShItem` such that `SHGetFileInfo` is not called until the property values that it provides are actually requested. This was done similarly to changes in a previous update that deferred the fetching of `IconIndex`es until actually requested.
- 11/05/2004 -- Update to the `CShItem` source. This fixes the following problems:
    - A memory leak in the `GetContents` method.
    - Changed the fetching of `IconIndex`es so that they are not obtained until called for. This should make little difference to applications that need icons for all files, but significantly speeds up applications that do not use or need icons for most files.
    - Fetch the correct icon for Open folders. It was finding and using the icon for *MyDocuments* for this purpose rather than the correct one.
    - Source download contains the correct code to match the article. This was not posted correctly for the last update prior to this one.
- 10/22/2004 -- Noticed that `SystemImageListManager`'s inappropriate design as a class with potential multiple instances caused problems, especially within the IDE, under some circumstances. Recast it as a class with only `Shared` properties and methods, as it should have been designed in the first place. With this change, and with the addition of a Mutex around the code that actually writes to the System image list, the class should be ThreadSafe, though that has not been tested exhaustively.
- 10/20/2004 -- Second version. Correct display of alpha channel icons on XP systems. One, simplified and corrected, class for managing icons. Revision of article to reflect code changes.
- 10/11/2004 -- Initial version of article and ExpTreeLib (Ver. 1.0.1743.41270).

This article was originally posted at http://www.codeproject.com/Articles/8546/An-All-VB-NET-Explorer-Tree-Control-with-ImageList

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Jim Parsells

United States

After 30+ years working in the IT field, mostly managing SysAdmins, I have retired. One of my hobbies returns me to programming, basically just to keep my hand in.

Watch

Comments and Discussions

Add a Comment or Question

Email Alerts

Search Comments

First Prev Next

How do I use the other "display mode"?
Member 14568147 13-Jul-20 18:08

Adding a drive manually
Ammar Khalil 24-May-18 6:05

Declaration Expected error
Rcomputer 2-May-18 13:11

Re: Declaration Expected error
Rcomputer 2-May-18 13:18

Dropbox sync overlay icons missing
Member 2492634 2-Aug-17 22:15

BeforeChange Event
Martin Gossmann 1-Mar-17 8:27

Re: BeforeChange Event
Martin Gossmann 2-Mar-17 8:54

Where can I get the latest version
Member 565540 8-May-16 12:38

Re: Where can I get the latest version
Jim Parsells 8-May-16 23:13

The path is not of a legal form

Tim Sines, CPA 7-Jan-16 18:52

Re: The path is not of a legal form
Tim Sines, CPA 7-Jan-16 18:58

Re: The path is not of a legal form
Jim Parsells 8-Jan-16 0:11

CSHItem Path on USB device (smartphone)
renator68 20-Oct-15 10:32

Re: CSHItem Path on USB device (smartphone)
Jim Parsells 20-Oct-15 12:22

StrRetToBuf AccessViolationException
DavidSparc 12-Sep-15 7:56

Re: StrRetToBuf AccessViolationException
Jim Parsells 14-Sep-15 0:19

extension?
ADemontis 17-Jun-15 18:47

Re: extension?
Jim Parsells 17-Jun-15 23:03

Re: extension?
ADemontis 18-Jun-15 0:53

Re: extension?
Jim Parsells 18-Jun-15 11:40

Re: extension?
ADemontis 18-Jun-15 12:41

Re: extension?
Jim Parsells 18-Jun-15 13:07

Re: extension?
Jim Parsells 18-Jun-15 12:15

Re: extension?
Jim Parsells 18-Jun-15 12:43

Re: extension?
ADemontis 18-Jun-15 13:01

Refresh 1 2 3 4 5 6 7 8 9 10 11 Next ▷

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2004 by Jim Parsells
Everything else Copyright © CodeProject, 1999-2022

Web01 2.8:2022-10-01:1